
B-Tree Leaf Node Format

Giving a page the shape of a tree node

Building a Simple Database • Part 8

A friendly, concept-first walk-through

Language: C • Written for readers newer to C

“A big change is best taken one small node at a time.”

June 19, 2026

Contents

1	Why a Tree At All?	2
2	One Node, One Page	2
3	The Header: What Every Node Records	2
3.1	Why we lay out bytes by hand	3
4	The Leaf Node, Field by Field	3
5	Reaching Into the Page	4
6	The Pager and Table Grow Up	4
7	The Cursor, Re-pointed	5
8	Inserting Into a Leaf	5
9	Seeing It Work	5
10	Still Unsorted, and That's Fine	6

1 Why a Tree At All?

So far our table has been the simplest thing that could work: an array of rows, one after another, in the order they were inserted. That format has real virtues. Each page holds nothing but row data, so it is space efficient, and inserting is instant because we just append to the end. But it has two matching weaknesses. Finding a particular row means scanning the whole table from the front, because nothing is sorted. And deleting a row would leave a hole that we would have to fill by shuffling every row after it backwards.

We could keep the array but hold the rows *sorted by id*. Then a lookup becomes a binary search, which is fast. But now insertion is the slow one, because slotting a new row into the middle means shifting everything after it to make space.

The tree is the structure that refuses to pick a loser. Each node is a page that holds a handful of rows plus a little bookkeeping, and the nodes are linked so that searching, inserting, and deleting all walk only a short path from the top of the tree to the bottom. We pay for this with a larger file — there is now metadata in every node, and whole internal nodes that hold no rows at all — but in return every important operation becomes fast.

	Unsorted array	Sorted array	Tree of nodes
Pages contain	only data	only data	metadata, keys, and data
Rows per page	more	more	fewer
Insertion	$O(1)$	$O(n)$	$O(\log n)$
Deletion	$O(n)$	$O(n)$	$O(\log n)$
Lookup by id	$O(n)$	$O(\log n)$	$O(\log n)$

This part does not build the whole tree — that takes several articles. By the end of this one we will have defined the exact byte layout of a single *leaf* node and be able to insert key/value pairs into a tree that consists of just that one node.

2 One Node, One Page

The rule that ties everything together: **each node of the tree is exactly one page**. When the tree code wants a node, it asks the pager for a page number and gets back a pointer into the page cache, exactly the machinery we already built. Internal nodes will point to their children simply by storing the child's page number; following a pointer means asking the pager for that page.

There will eventually be two kinds of node, and they have different layouts, so we need a way to tell them apart. In C this is what an *enum* is for: a named list of integer constants. Declaring the two node types as an enum gives us the readable names `NODE_INTERNAL` and `NODE_LEAF` instead of bare numbers like 0 and 1. This part only deals with leaf nodes — the ones that actually hold rows — but we name both now because the type will be the very first thing stored in every node.

3 The Header: What Every Node Records

A node cannot be just raw rows any more; it needs a little information about itself, stored in a *header* at the start of the page. Every node, whatever its kind, records three things in a common header:

- its **node type** — leaf or internal — stored in a single byte,
- whether it is the **root** of the tree, also a single byte used as a true/false flag,

- a **parent pointer** — the page number of its parent, four bytes — so that from any node we can find its way back up and reach its siblings.

That is six bytes in total: one, plus one, plus four.

3.1 Why we lay out bytes by hand

Here is a point worth pausing on if you are newer to C. You might expect us to describe a node with a C `struct`, the way we described a `Row`. Instead we describe the layout with a set of *offset and size constants* — “the node type is one byte at offset 0; the root flag is one byte at offset 1; the parent pointer is four bytes at offset 2” — and then reach into the page by adding those offsets to the page’s address.

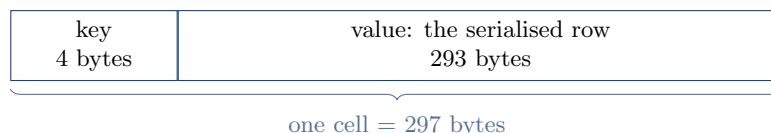
The reason is that this layout is written to a file and must be exactly the same every time, on every machine. A C `struct` does not give that guarantee: the compiler is allowed to insert invisible *padding* between fields for alignment, and the amount can differ between compilers. By computing the byte positions ourselves, we control the format precisely, which is what an on-disk format demands. The constant `sizeof(uint8_t)` is just C’s way of saying “the size of a one-byte unsigned integer,” i.e. 1, and `sizeof(uint32_t)` is 4; we build the offsets out of these so that if a field’s size ever changes, the offsets after it adjust automatically.

It is, as the original author notes, a little wasteful to spend a whole byte on a single true/false value, but it keeps the code that reads and writes those fields simple, which is a fair trade at this stage.

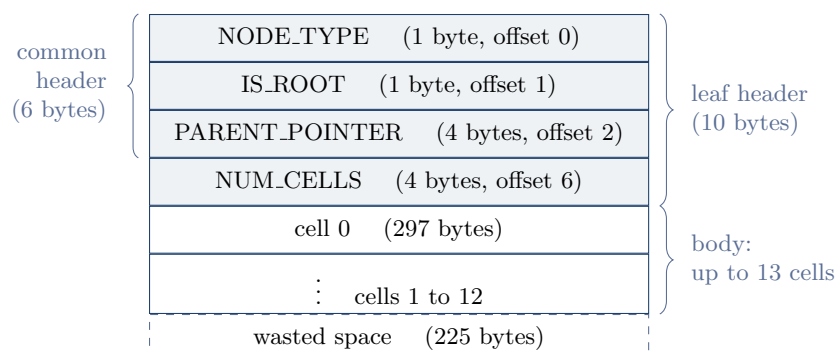
4 The Leaf Node, Field by Field

A leaf node is the kind that stores actual rows. On top of the common six-byte header, it records one more number: how many *cells* it currently holds, stored as a four-byte value. A cell is simply a key paired with a value — the key being the row’s id, and the value being the serialised row itself. So the full leaf header is ten bytes: the six common ones plus the four-byte cell count.

After the header comes the body, which is just an array of cells laid end to end. Each cell is a four-byte key followed by the row’s bytes. With our row taking 293 bytes, a cell is $4 + 293 = 297$ bytes.



How many cells fit? The page is 4096 bytes; subtract the ten-byte header and 4086 bytes remain for cells. Dividing by 297 gives 13 whole cells, with 225 bytes left over. We deliberately leave that remainder empty rather than cramming in a partial fourteenth cell, because a cell must never be split across two nodes. Here is the whole page:



5 Reaching Into the Page

Because a node is just raw bytes, we need small helper functions that, given a page, hand back a pointer to a particular field inside it. They are all simple pointer arithmetic: take the page's address and add the right offset.

There is a neat C idiom at work here. Each helper returns a *pointer* to the field rather than the field's value. A pointer can be used two ways: you can read through it to get the value, or you can assign through it to set the value. So a single function serves as both getter and setter. Writing “the number of cells, plus one” back into the header is then just dereferencing the pointer and adding one — no separate set-function needed.

Helper	What it returns
<code>uint32_t* leaf_node_num_cells(void*)</code>	A pointer to the cell-count field in the header. Read it to ask how many cells the node holds; assign through it to change that count.
<code>void* leaf_node_cell(void*, uint32_t)</code>	A pointer to the start of cell number <i>n</i> : skip the ten-byte header, then step over <i>n</i> whole cells.
<code>uint32_t* leaf_node_key(void*, uint32_t)</code>	A pointer to cell <i>n</i> 's key. Since the key sits first in a cell, this is just the cell's start address.
<code>void* leaf_node_value(void*, uint32_t)</code>	A pointer to cell <i>n</i> 's value: the cell's start plus the four-byte key.
<code>void initialize_leaf_node(void*)</code>	Set the cell count to zero, producing a fresh, empty leaf.

The `void*` you see is C's “pointer to untyped memory” — just an address, with no opinion about what lives there. The page is exactly that: a block of bytes we interpret ourselves. Where a helper returns `uint32_t*` instead, it is promising that the four bytes at that address should be read as a number.

6 The Pager and Table Grow Up

This new format lets us delete an awkward piece of older code. Back in the persistence part, the last page of the file could be partial, so flushing a page needed to know how many bytes to write. Now that **every node fills exactly one page**, that complication vanishes: a flush always writes a full page, and the size argument disappears entirely. The partial-page special case we admitted was inelegant is now gone for good.

A few related adjustments come with it:

- The pager now tracks `num_pages`, the number of pages in the database. This is a property of the file as a whole, so it belongs to the pager. When the pager opens the file it computes this from the file length, and it now insists the file be a whole number of pages — if it is not, the file is corrupt and we stop.
- Whenever `get_page` hands out a page number it has never seen before, it bumps `num_pages` so the count stays current.
- Closing the database now loops over `num_pages` and flushes each page whole, with no leftover-rows special case to worry about.
- The table no longer stores `num_rows`. A tree is identified by its **root node's page number**, so the table now keeps `root_page_num` instead. “How many rows” is no longer a single stored number; it is something you would discover by walking the tree.

7 The Cursor, Re-pointed

In the cursor part, a position in the table was a single row number, because the table was a flat list. A tree has no such global row number. A position is now two coordinates: *which node* (its page number) and *which cell within that node*. So the cursor swaps its lone `row_num` for a `page_num` and a `cell_num`.



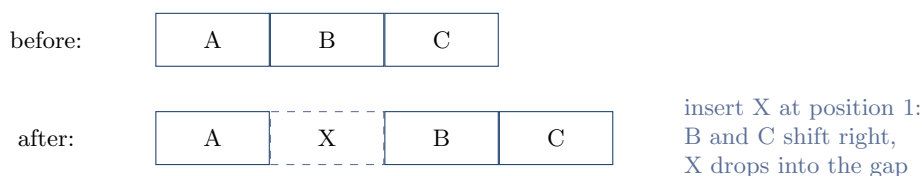
The four cursor functions adapt accordingly, but their *shape* is exactly what we designed it to be in the previous part — which is the whole reason that refactor was worth doing. To start a cursor, point it at the root page, cell zero, and mark it end-of-table only if the root has no cells. To make an end cursor, point it at the root page and set its cell number to the node's cell count. To read a value, ask the pager for the cursor's page and return a pointer to that cell's value. To advance, add one to the cell number and, if it reaches the node's cell count, mark the end. Notice that each of these now reads the cell count out of the node's header using the helper from earlier, rather than from a table-wide counter.

8 Inserting Into a Leaf

When the database file is brand new it is completely empty, so there is no root node yet. On opening, if the pager reports zero pages, we create page 0 and initialise it as an empty leaf — that becomes the root. From then on the file always has at least its root node.

Inserting a key/value pair into a leaf is handled by one new function that takes a cursor (the position to insert at), the key, and the row. It does three things:

1. **Check for room.** If the node already holds the maximum number of cells, it is full. Splitting a full leaf into two is a later part, so for now we simply report that splitting is not yet implemented and stop.
2. **Make room if needed.** If the insert position is before the end of the existing cells, every cell from the insertion point onward is copied one slot to the right, working from the back forwards so nothing is overwritten before it is moved. This opens a gap at the insertion point.
3. **Write the new cell.** Increase the cell count by one, write the key into the gap, and serialise the row into the value slot beside it.



Because the tree currently has only one node, the part of the code that runs an insert simply checks whether the root leaf is full and, if not, calls this helper through a cursor positioned at the end of the node.

9 Seeing It Work

Two small meta-commands let us inspect what we have built. One prints the layout constants, which is handy both for understanding and as a test that alerts us if a size ever changes:

Constant	Value	Meaning
ROW_SIZE	293	bytes in one serialised row
COMMON_NODE_HEADER_SIZE	6	type + root flag + parent pointer
LEAF_NODE_HEADER_SIZE	10	common header + cell count
LEAF_NODE_CELL_SIZE	297	key (4) + value (293)
LEAF_NODE_SPACE_FOR_CELLS	4086	page (4096) minus header
LEAF_NODE_MAX_CELLS	13	how many cells fit in one leaf

So a single leaf holds at most thirteen rows for now. The other command prints the tree: for our one-node tree it reports the leaf's size and lists each cell's position and key.

10 Still Unsorted, and That's Fine

There is an honest wrinkle to admit. Inserts still go in through a cursor positioned at the *end* of the node, so rows are stored in the order they were typed, not sorted by key. If you insert ids 3, 1, then 2 and print the tree, you will see them in the order 3, 1, 2 — exactly as before. The shift-to-make-room logic is in place, but nothing yet calls for an insert in the middle, so it never fires.

This can feel like a step backwards: the database now holds fewer rows than it used to, and they are still unsorted. But that is the nature of a large change broken into safe steps. We have given our pages the shape of tree nodes and taught the pager, the table, and the cursor to speak in nodes and cells instead of flat row numbers. The structure is now in place to do the genuinely new work.

Next time: finding a row by its key with binary search, and finally storing rows in sorted order.